TECHNICAL REPORT

# QuizTime Online Exam System

Detailed System Documentation: Functional & Non-Functional Specifications

An enterprise-grade online testing portal featuring automated evaluation, dynamic question distribution, customized grading limits, and comprehensive browser and webcam-based proctoring mechanisms.

| | |
|---|---|
| **Prepared For:** | ICBT BSC Degree Programme - Development Project |
| **System Version:** | v1.0.0 (Production-Ready) |
| **Date:** | June 05, 2026 |

QuizTime Group

# Table of Contents

# 1. Executive Summary & System Overview

In modern education and corporate training, online assessment platforms play an essential role. However, maintaining the integrity, flexibility, and reliability of remote exams remains a key challenge. **QuizTime** is a state-of-the-art Online Exam System designed to address these concerns.

QuizTime provides a fully decoupled, double-sided portal system separating administrative features from student evaluation environments. It enables educators to upload extensive question banks, schedule highly customized exams, automatically generate student-specific question sheets, and track live statistics. Meanwhile, it offers students a distraction-free exam terminal backed by live webcam stream tracking and active browser focus lockdowns to prevent academic dishonesty.

Key innovations in QuizTime include:

- **Dual-Mode Timers:** Supports standard test-wide global limits or strict per-question limits to enforce testing speed.
- **Dynamic Allocation:** Allows randomized selection rules from distinct question categories so no two students take the exact same test.
- **Active Client Proctoring:** Monitors camera state and tab focuses, triggering auto-submissions when policies are violated.
- **Network-Resilient State Sync:** Employs continuous progress syncing and background beacons to ensure no student progress is lost during network dropouts or accidental window closures.

# 2. Architecture & Technology Stack

The system is designed on a modular, layered REST architecture, separating user interaction, business logic processing, and transactional database storage.

REACT + VITE VANILLA CSS SPRING BOOT JAVA 21 MYSQL 8.0 SPRING SECURITY CRYPTO

### A. User Interface (Frontend Portal)

The frontend runs as a Single Page Application (SPA) compiled using **Vite**. It uses **React** for component-driven UI rendering and state management. The layout uses custom **Vanilla CSS** to provide a modern, responsive design characterized by glassmorphism, smooth transitions, and distinct typography. **Lucide React** supplies vector icon assets. Camera feeds are processed using **React-Webcam**.

### B. Business Logic Layer (Backend API)

The server layer is built using **Spring Boot (Java 21)**, structured into standard layers:

- **Controllers:** Expose clean, structured REST API endpoints for user authentication, CRUD data management, test verification, and exam portals.
- **Services:** Contain core transaction logic, including automatic grading computations, question randomization algorithms, email template formatting, and scheduling background jobs.
- **Repositories:** Manage relational query executions mapped through **Spring Data JPA (Hibernate)**.

### C. Persistence Layer (Relational Storage)

Data is stored in a **MySQL 8.0** relational database. Tables are structured with cascading delete constraints and indexes (e.g., matching unique constraints on composite columns like test/student identifiers) to ensure referential integrity and optimized read queries.

# 3. Functional Specifications

The platform splits operations into two portals: the Administrator Dashboard and the Student Portal.

## 3.1 Administrator Portal

The Admin Panel provides complete control over the system's database entities and exam execution cycles:

- **Dashboard Analytics:** Renders visual panels capturing total student registrations, question counts by categories, active tests in progress, overall average score distributions, and real-time pass/fail rates.
- **Student Management:** Provides CRUD controls to search, add, edit, or delete students. Supports bulk student import via CSV files. The system retains a detailed historical status history for each student.
- **Question Pool:** Supports categorization (e.g., Math, General Science) and manages questions. Each question includes text, type (MCQ or Short Answer), options list (if MCQ), correct answer key, and active/inactive status. Supports CSV bulk imports.
- **Test Scheduler:** Allows configuration of exam attributes:
  - *Selection Mode:* Manual picker (fixed list) or Dynamic configuration (e.g., dynamically pull X random questions from category Y).
  - *Time Mode:* Set a global timer (e.g., 60 minutes) or a per-question timer (e.g., 60 seconds per question).
  - *Navigation Layout:* Scroll mode (all questions shown) or Step mode (must answer one question at a time).
  - *Display Rules:* Toggle whether scores/grades are displayed immediately, and whether correct answers are revealed after submission.
- **Rescheduling & Adjustments:** Allows admins to grant individual time extensions (Additional Time in minutes) or reopen submitted tests for students.
- **Grading Rules:** Admins define custom letter grades (A, B, C, F), GPA scales, and minimum percentage ranges.
- **System Setup:** Allows editing of SMTP host settings (SMTP user, password, port, and security certificates) for emailing reports.

## 3.2 Student Portal

The Student Portal is optimized for performance, security, and low cognitive load:

- **Code Check-In:** Students enter their name and a unique 4-character exam code generated by the admin to check in. The system verifies access restrictions (e.g., whether the code has expired or is already in use).
- **Rules Overlay:** Displays exam guidelines, question count, and time limits before launching the secure workspace.
- **Assessment Interface:** Renders the scheduled exam configuration. Includes interactive radio options for MCQs, text inputs for short answers, navigation panels to jump between questions, flags to mark questions for review, and a real-time countdown timer.
- **Score & Review Panels:** If configured by the admin, students receive immediate grading feedback, letter grades, and question-by-question correctness breakdowns upon submission.

# 4. Academic Integrity & Security Infrastructure

To prevent cheating, QuizTime combines browser APIs with background processes to enforce a locked testing environment:

> **Proctoring Rule & Strike Enforcement**
>
> Any action violating the strict exam configurations (tab switching, stream loss) is treated as an infraction. If a student reaches **3 warnings**, the system disables input and automatically submits the exam to prevent further tampering.

### 4.1 Multi-Tab & Multi-Session Protection

Students cannot open the exam in multiple tabs or browsers concurrently. The portal instantiates a client-side `BroadcastChannel` bound to the unique exam code. When a tab loads, it broadcasts a verification ping. If an active tab receives this ping, it triggers an access-denial message back to the new tab, which immediately locks itself out and clears active local session tokens.

### 4.2 Tab Switching Lockout & Warnings

The portal registers a listener on the HTML5 `visibilitychange` API. If a student switches tabs, minimizes their browser, or clicks away to an external application, the browser triggers a warning modal. If the student triggers this 3 times, the interface runs an automated submit request, saving their responses and terminating the session.

### 4.3 Webcam Proctoring Monitor

The exam environment integrates a live `WebcamProctor` component. It checks for webcam permissions before initiating the test and renders a live preview in the viewport. The component runs background checks every 5 seconds. If the camera stream is disabled, blocked, or disconnected, the system logs an infraction message.

### 4.4 Ephemeral Progress Sync & Recovery

To mitigate data loss from network issues or browser crashes:

- **Session Mirroring:** All answers are saved locally in the browser's `SessionStorage` instantly upon interaction.
- **Backend Synchronization:** The client triggers asynchronous API synchronization requests regularly to save student answers and elapsed times in the database.

- **Tab Close Recovery (Beacon):** If a student closes the tab or window, the system uses the `navigator.sendBeacon()` API to send a background payload to the backend, ensuring their progress is saved.
- **Resumption Engine:** If a student restarts their computer, they can re-enter their code to restore their answers and continue the exam with their remaining time.

# 5. Non-Functional Specifications

## 5.1 Security & Access Controls

- **Credential Protection:** Passwords in the database are hashed using the `BCryptPasswordEncoder` from Spring Security.
- **One-Time Access Tokens:** Student exam codes are single-use 4-character tokens. Once used, their status is updated to `USED`, blocking future entry.

## 5.2 Reliability & Fault-Tolerance

- **Retry Logic:** When submitting the final exam sheet, if the network request fails, the application automatically retries the submission up to 3 times, waiting 1 second between attempts, before alerting the user.
- **Transaction Isolation:** Database operations in the service layer are annotated with `@Transactional` to guarantee that database updates rollback in the event of an mid-execution error.

## 5.3 Performance & Optimization

- **Database Query Optimization:** The backend entities configure Hibernate `@BatchSize(size = 100)` to optimize database performance and prevent "N+1 query" bottlenecks.
- **GZIP Compression:** Spring Boot properties enable response compression for key MIME types (HTML, CSS, JS, JSON) larger than 1KB, reducing payload transfer sizes and latency.

## 5.4 Usability & UI Aesthetics

- **Modern Interface:** Styled with a dark mode base, glassmorphism containers, smooth hover states, and distinct badges to provide a premium user experience.
- **Keyboard Friendly:** Features accessible layouts, and mouse navigation to assist students during long assessments.

# 6. Relational Database Schema Design

The MySQL database schema is structured to ensure fast read-write access and data normalization:

| Table Name | Primary Key | Key Fields & Types | Relationships & Explanations |
|---|---|---|---|
| `admins` | `id` (BIGINT) | `username` (VARCHAR), `password` (VARCHAR) | Stores administrator authentication details. |
| `students` | `id` (BIGINT) | `name` (VARCHAR), `nic` (VARCHAR), `status` (VARCHAR) | Tracks student records and exam enrollment statuses. |
| `questions` | `id` (BIGINT) | `text` (TEXT), `type` (VARCHAR), `correct_answer` (VARCHAR) | Mapped to `categories` via a foreign key (Category ID). Mapped to many-to-many test tables. |
| `question_options` | Composite | `question_id` (FK), `option_text` (VARCHAR) | Stores possible options for MCQ questions. Mapped via collection tables. |
| `categories` | `id` (BIGINT) | `name` (VARCHAR), `status` (VARCHAR) | Grouping directories for dividing question pools. |
| `tests` | `id` (BIGINT) | `name` (VARCHAR), `time_mode` (VARCHAR), `exam_mode` (VARCHAR) | Contains core test variables. Has one-to-many configurations with category limits. |
| `student_exam_codes` | `id` (BIGINT) | `exam_code` (VARCHAR), `status` (VARCHAR), `test_id` , `student_id` | Holds unique, temporary student-test access codes. Unique constraint on `(test_id, student_id)` . |
| `submissions` | `id` (BIGINT) | `score` (INT), `answers_json` (LONGTEXT), `detailed_breakdown_json` (LONGTEXT) | Maintains final assessment scores and question-by-question metrics. |
| `grading_scales` | `id` (BIGINT) | `grade_name` (VARCHAR), `min_percentage` (INT) | Grade definitions used to assign final grades. |

# 7. Conclusion & Next-Phase Roadmap

The **QuizTime Online Exam System** is a comprehensive, production-ready solution that combines a highly configurable backend with a secure testing interface. Its features, such as automated grading, dynamic question allocation, webcam and browser-based proctoring, and session recovery, provide a robust platform for modern assessments.

To further improve the system in future phases, we recommend the following enhancements:

- **AI Face-API.js Integration:** Implement browser-side face recognition to verify student identity throughout the exam and flag multiple faces or student absence.
- **Websocket-based Live Proctoring Dashboard:** Allow administrators to monitor active student camera streams, warning counts, and progress metrics in real-time from their dashboard.
- **Rich Text & Code Editor Questions:** Expand the question bank to support complex coding environments and rich text submissions for programming or essay assessments.